

파이썬 기초 문법의 학습 흐름에 맞춰 프로그램이 어떻게 '진화'할 수 있는지 보여주는 4단계 로드맵입니다. 문법을 배울 때마다 내 프로그램이 똑똑해지는 것을 체감하기에 아주 좋은 설계 방식입니다.

## 1단계: 리스트 (데이터를 담는 바구니 만들기)

단순한 변수(`price = 50000`)를 넘어, 여러 개의 데이터를 한곳에 모아두는 법을 배우는 단계입니다.

- 구현 기능: 보유 종목명, 매수 단가, 보유 수량을 각각의 리스트에 저장합니다.
- 프로그램의 모습:
- `names = ["삼성전자", "네이버", "카카오"]`
- `prices = [75000, 200000, 50000]`
- `amounts = [10, 5, 20]`
- 이 단계의 한계: "삼성전자"의 정보를 찾으려면 리스트 3개의 첫 번째 칸을 일일이 확인해야 합니다. 아직 스스로 판단을 내리거나 반복적인 작업을 하지는 못하는 단순 기록장 수준입니다.

---

## 2단계: 조건문 (프로그램에 판단력 부여하기)

`if/elif/else`를 통해 데이터에 규칙을 부여하고, 기계적인 결정을 내리게 만드는 단계입니다.

- 구현 기능: 손절/익절 알림 및 비중 경고 시스템을 추가합니다.
- 프로그램의 모습:
- 지정한 현재가를 바탕으로 수익률을 계산합니다.
- `if` 수익률 < -10: 이면 "손절매 권장" 메시지를 출력합니다.
- `elif` 특정 종목 투자금 > 1000000: 이면 "투자 비중 과다" 경고를 띄웁니다.
- 이 단계의 발전: 나만의 투자 원칙을 코드로 이식했습니다. 하지만 여전히 데이터를 한 번만 검사하고 프로그램이 바로 종료되는 단발성 구조입니다.

---

## 3단계: 반복문 (자동화와 계속 켜져 있는 프로그램)

`for`와 `while`을 활용해 반복 작업을 기계에게 맡기고, 프로그램이 종료되지 않게 만드는 핵심 단계입니다.

- 구현 기능: 메인 메뉴(UI) 생성 및 포트폴리오 일괄 브리핑 기능을 추가합니다.
- 프로그램의 모습:
- `while True`: 문을 사용해 "1. 주식 추가, 2. 수익률 확인, 3. 종료" 형태의 무한 루프 메뉴판을 만듭니다. 사용자가 3번을 누를 때까지 프로그램은 계속 실행됩니다.
- `for` 문을 사용해 `names` 리스트에 있는 모든 주식의 현재 수익률을 1번부터 끝번까지 단숨에 계산하고 화면에 짝 출력해 줍니다.
- 이 단계의 발전: 드디어 우리가 아는 '소프트웨어'다운 형태를 갖췄습니다. 하지만 코드가 점점 길어지고, 1단계에서 만든 병렬 리스트(리스트 3개)를 관리하는 것이 벅차지기 시작합니다.

---

## 4단계: 함수와 딕셔너리 (전문가 수준의 구조화)

지저분해진 코드를 깔끔하게 정리하고(def), 흩어진 데이터를 의미 있게 묶는(dict) 단계입니다.

- 구현 기능: 종목별 데이터 통합 및 반복되는 계산 로직의 모듈화입니다.
- 프로그램의 모습:
- 딕셔너리 도입: 이름, 가격, 수량 리스트를 따로 두지 않고 하나로 묶습니다. `portfolio = [{"name": "삼성전자", "price": 75000, "amount": 10}]` 형태로 정리되어 데이터 관리가 압도적으로 편해집니다.
- 함수 도입: 수익률을 계산하는 복잡한 수식을 `def calculate_yield(buy, current):` 라는 함수로 빼냅니다. 이제 코드가 복잡해 보이지 않고, 필요할 때마다 이 함수를 부르기만 하면 됩니다.
- 이 단계의 완성: 코드가 깔끔해져서 나중에 기능을 추가(예: 파일 저장 기능, 그래프 그리기 등)하기가 매우 쉬워집니다. 어엿한 포트폴리오 매니저 프로그램이 완성되었습니다.

---

이 4단계 로드맵 중에서 현재 가장 먼저 코드로 구현해 보고 싶은 단계는 어디인가요?

원하신다면 해당 단계의 뼈대가 되는 아주 간단한 파이썬 예시 코드를 보여드릴 수 있습니다.